

UC San Diego

Electrical and Computer Engineering
JACOBS SCHOOL OF ENGINEERING



Denoising Diffusion Models

ECE 285 Deep Generative Models

Youwei Liang

Today's topic

- Denoising Diffusion Probabilistic Models (DDPM)
 - Rigorous formulation
- Deriving DDPM's objective
- Implementation details

Recap: diffusion (forward) process

- Gradually add Gaussian noise to an image
- Image $\rightarrow$ Gaussian noise
 - Histogram of pixel values is a Gaussian distribution



- Noising process is a Markov chain
 - “Shrink” the image pixels
 - Perturb each pixel by a random value
 - The step sizes are controlled by a variance schedule: $\{\beta_t \in (0, 1)\}_{t=1}^T$

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

$$q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$$

Conditional distribution

- Can we directly get $\mathbf{x}_t$ from $\mathbf{x}_0$?

- Compute the conditional $q(\mathbf{x}_t|\mathbf{x}_0)$
- Reparameterization trick

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$\text{Let } \alpha_t = 1 - \beta_t \text{ and } \bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}_{t-1} \quad \text{where } \boldsymbol{\epsilon}_{t-1}, \boldsymbol{\epsilon}_{t-2}, \dots \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}}\boldsymbol{\epsilon}_{t-2} \right) + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}_{t-1}$$

$$= \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{\alpha_t}\sqrt{1 - \alpha_{t-1}}\boldsymbol{\epsilon}_{t-2} + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}_{t-1}$$

$$= \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\boldsymbol{\epsilon}}_{t-2}$$

$$= \dots$$

$$= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$$

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

Adding two Gaussian

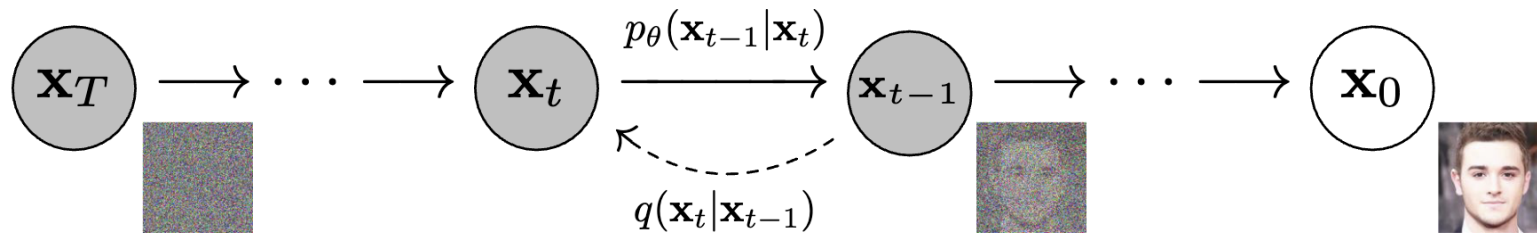
$$\mathcal{N}(\mathbf{0}, \sigma_1^2\mathbf{I}) \text{ and } \mathcal{N}(\mathbf{0}, \sigma_2^2\mathbf{I}) \Rightarrow \mathcal{N}(\mathbf{0}, (\sigma_1^2 + \sigma_2^2)\mathbf{I})$$

- “Shrink” the image and randomly perturb each pixel
- $\bar{\alpha}_t \rightarrow 0$ when t becomes large

DDPM: Reverse process

- Start from Gaussian noise, gradually “denoise”
- The reverse conditional $q(x_{t-1}|x_t) = q(x_t|x_{t-1})q(x_{t-1})/q(x_t)$
 - Intractable due to $q(x_t)$
- Need to learn $p(x_{t-1}|x_t)$
 - Design choice (Gaussian): $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$

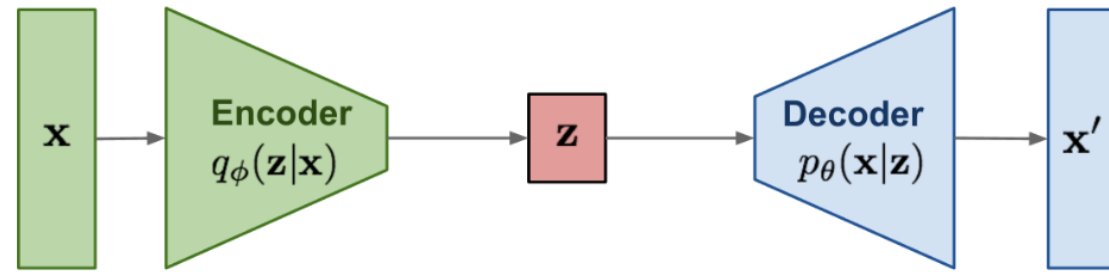
$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$



Connection to VAE

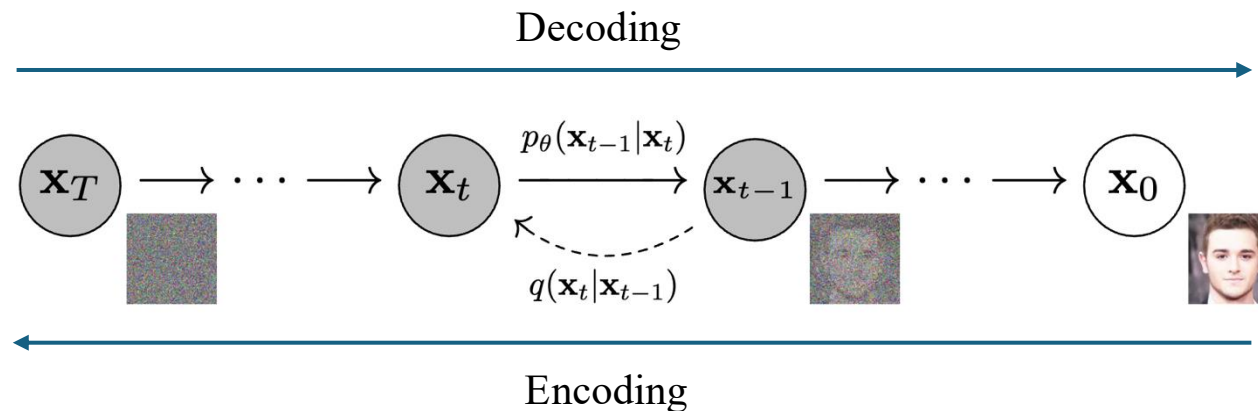
- VAE

- Convert an image to a latent variable z and then back to an image
- Learnable encoder and decoder
- ELBO objective: $\log p_{\theta}(\mathbf{x}^{(i)}) \geq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [-\log q_{\phi}(\mathbf{z}|\mathbf{x}) + \log p_{\theta}(\mathbf{x}, \mathbf{z})]$



- DDPM as VAE

- Convert an image to a set of latent variables $z = \{x_1, x_2, \dots, x_T\}$ and then back to an image
- Encoder is not learned—diffusion process
- Decoder is learned to convert $\{x_1, x_2, \dots, x_T\}$ to image x_0



Learn DDPM as VAE

- ELBO for VAE: $\log p_{\theta}(\mathbf{x}^{(i)}) \geq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [-\log q_{\phi}(\mathbf{z}|\mathbf{x}) + \log p_{\theta}(\mathbf{x}, \mathbf{z})]$

$$\min_{\theta} L = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log q_{\phi}(\mathbf{z}|\mathbf{x}) - \log p_{\theta}(\mathbf{x}, \mathbf{z})]$$

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{x}, \mathbf{z})}$$

$$= \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{0:T})}$$

$$= \mathbb{E}_q \left[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_{\theta}(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right]$$

$$= \mathbb{E}_q \left[-\log p_{\theta}(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right]$$

Monte Carlo
sampling of $\mathbf{x}_t$:
high variance

$$= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0) \prod_{t=2}^T q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right]$$

$$= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_{\theta}(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)} - \log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1) \right]$$

$$= \underbrace{C - \mathbb{E}_q[\log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1)]}_{L_0} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}}$$

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0)$$

$$= q(\mathbf{x}_{1:T-1}|\mathbf{x}_T, \mathbf{x}_0) \cdot q(\mathbf{x}_T|\mathbf{x}_0)$$

$$= q(\mathbf{x}_{1:T-2}|\mathbf{x}_{T-1}, \mathbf{x}_T, \mathbf{x}_0) \cdot q(\mathbf{x}_{T-1}|\mathbf{x}_T, \mathbf{x}_0) \cdot q(\mathbf{x}_T|\mathbf{x}_0)$$

$$= q(\mathbf{x}_{1:T-2}|\mathbf{x}_{T-1}, \mathbf{x}_0) \cdot q(\mathbf{x}_{T-1}|\mathbf{x}_T, \mathbf{x}_0) \cdot q(\mathbf{x}_T|\mathbf{x}_0)$$

$$= q(\mathbf{x}_T|\mathbf{x}_0) \prod_{t=2}^T q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$$

Derive q

- $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$

$$\begin{aligned}
 q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &= q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \\
 &= q(\mathbf{x}_t|\mathbf{x}_{t-1}) \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \\
 &\propto \exp \left(-\frac{1}{2} \left(\frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right) \right) \\
 &= \exp \left(-\frac{1}{2} \left(\frac{\mathbf{x}_t^2 - 2\sqrt{\alpha_t} \mathbf{x}_t \mathbf{x}_{t-1} + \alpha_t \mathbf{x}_{t-1}^2}{\beta_t} + \frac{\mathbf{x}_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0 \mathbf{x}_{t-1} + \bar{\alpha}_{t-1} \mathbf{x}_0^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right) \right) \\
 &= \exp \left(-\frac{1}{2} \left(\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right) \mathbf{x}_{t-1}^2 - \left(\frac{2\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right) \mathbf{x}_{t-1} + C(\mathbf{x}_t, \mathbf{x}_0) \right) \right)
 \end{aligned}$$

$$\begin{aligned}
 &ax^2 + bx + c \\
 &= a \left(x + \frac{b}{a} \right)^2 + c_2
 \end{aligned}$$

$$\begin{aligned}
 \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) &= \left(\frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right) / \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right) \\
 &= \left(\frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right) \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot \beta_t \\
 &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0
 \end{aligned}$$

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_t)$$

$$\begin{aligned}
 \tilde{\boldsymbol{\mu}}_t &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_t) \\
 &= \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_t \right)
 \end{aligned}$$

KL divergence

$$\underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}}$$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t, \tilde{\beta}_t \mathbf{I}) \quad \tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_t \right) \quad \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

- We *choose* to learn this parameterization: $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \beta_t \mathbf{I})$

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right)$$

$$D_{KL}(p||q) = \frac{1}{2} \left[\log \frac{|\Sigma_q|}{|\Sigma_p|} - k + (\boldsymbol{\mu}_p - \boldsymbol{\mu}_q)^T \Sigma_q^{-1} (\boldsymbol{\mu}_p - \boldsymbol{\mu}_q) + \text{tr} \{ \Sigma_q^{-1} \Sigma_p \} \right]$$

$$\begin{aligned} L_t &= \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{1}{2\tilde{\beta}_t^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{1}{2\tilde{\beta}_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_t \right) - \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) \right\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{(1 - \alpha_t)^2}{2\alpha_t(1 - \bar{\alpha}_t)\tilde{\beta}_t^2} \|\boldsymbol{\epsilon}_t - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{(1 - \alpha_t)^2}{2\alpha_t(1 - \bar{\alpha}_t)\tilde{\beta}_t^2} \|\boldsymbol{\epsilon}_t - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_t, t)\|^2 \right] \end{aligned}$$

Loss function

- Training is better without the weighting term

$$\begin{aligned} L_t^{\text{simple}} &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t)\|^2 \right] \end{aligned}$$

Algorithm

- Training
 - Sample Gaussian noise
 - Sample a time step
 - Scale the image and add the rescaled noise
 - Predict the noise
- Inference
 - Predict mean of the noisy image at the previous time step

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I}) \quad \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t)\|^2$ 
6: until converged
  
```

Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
  
```

Implementation

- Choosing diffusion steps T and noise magnitude β_t ($t = 1, \dots, T$)
 - Recall we use a Gaussian to approximate the true reverse conditional $q(x_t|x_{t-1})$
 - β_t needs to be small enough
 - Increasing linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I})$$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = q(\mathbf{x}_t|\mathbf{x}_{t-1}) \frac{q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)} \qquad q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \qquad \alpha_t = 1 - \beta_t \text{ and } \bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

- At the end of noising steps, we want $x_T \sim \mathcal{N}(0, I)$
 - $\bar{\alpha}_T$ should be close to 0
 - $T = 1000$

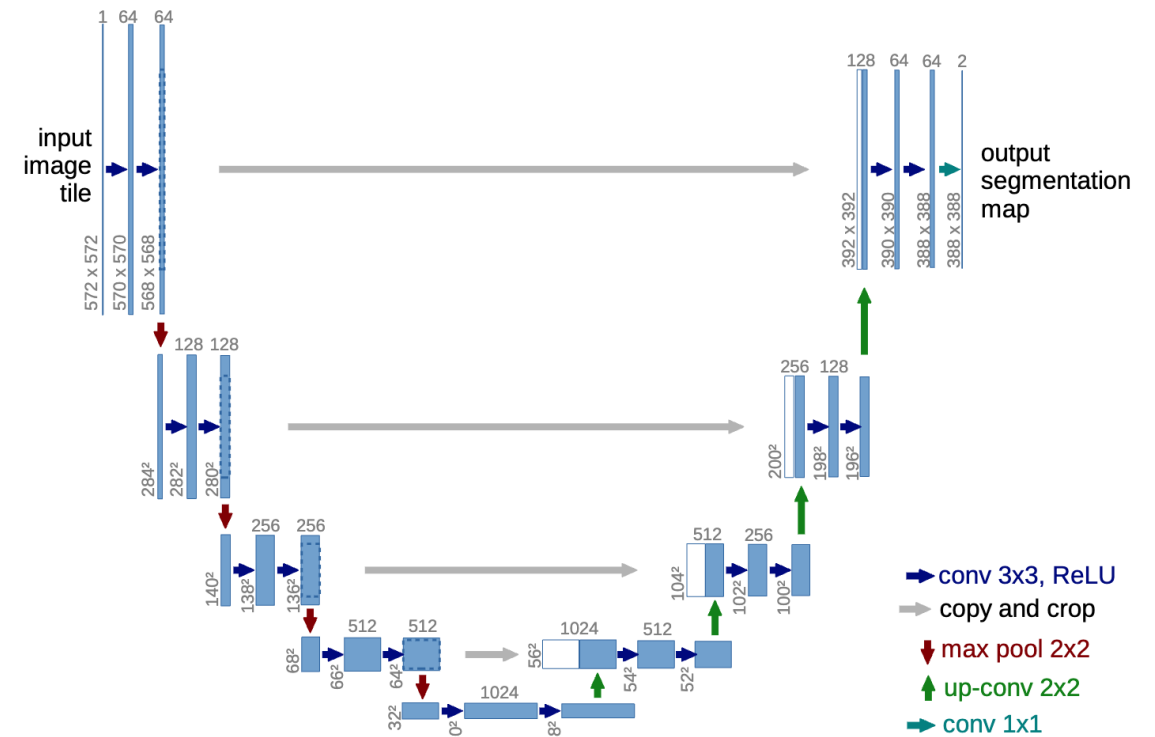
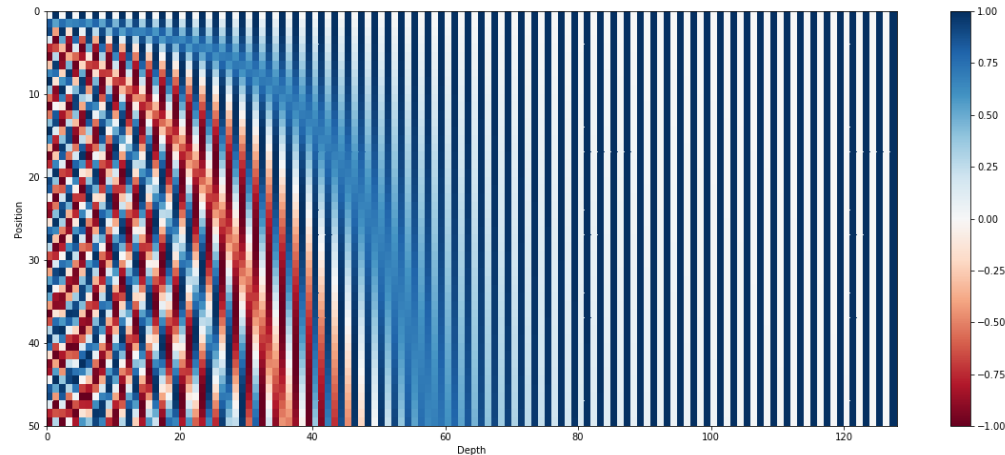
$$\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$$

Implementation: U-Net

- Noise prediction network
 - Input: noisy image x_t and time step t
 - Output: noise ϵ (same size as image)
- U-Net
 - Reduced spatial size in the middle
 - Skip connection from input to output
- Time step embedding
 - Transformer sinusoidal position embedding

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I})$$

$$\boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right)$$

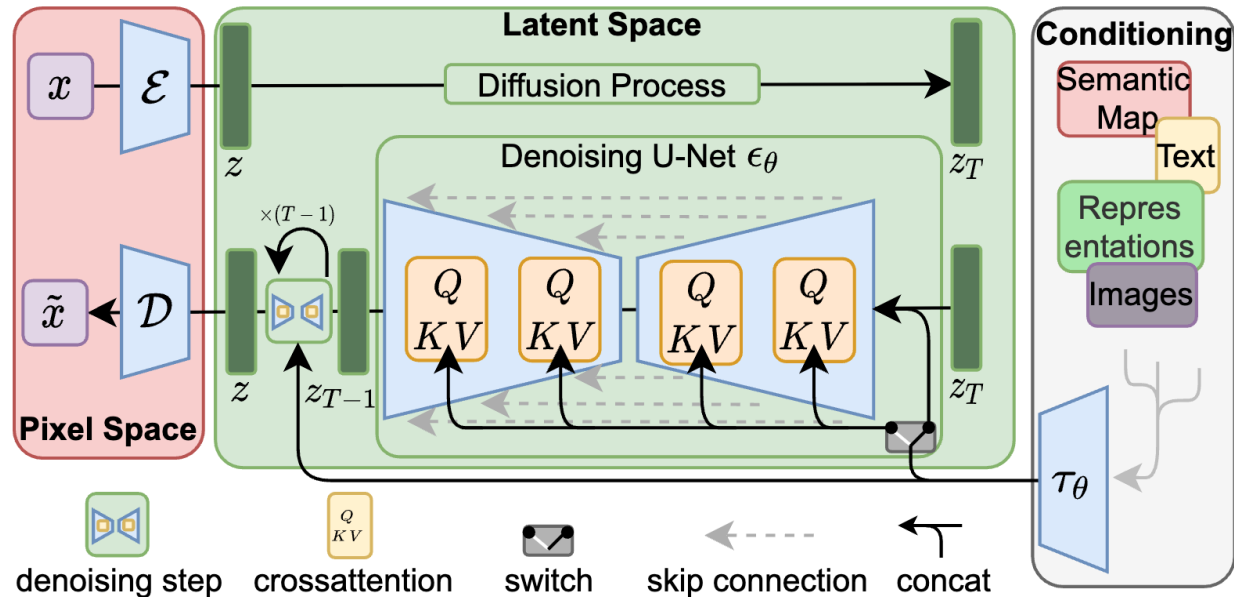


Implementation: Latent diffusion model (LDM)

- Forward process on image space is computationally expensive
 - Resolution: 1024x1024x3 $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$
- An encoder compresses the image to a latent space, and a decoder convert it back to image space
 - Use VAE for the encoder and decoder $\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{h \times w \times c}$ $\tilde{\mathbf{x}} = \mathcal{D}(\mathbf{z})$
- Diffusion and denoising processes happen on the latent vector $\mathbf{z}$

$$L_{LDM} := \mathbb{E}_{\mathcal{E}(x), \epsilon \sim \mathcal{N}(0,1), t} \left[\|\epsilon - \epsilon_{\theta}(z_t, t)\|_2^2 \right]$$

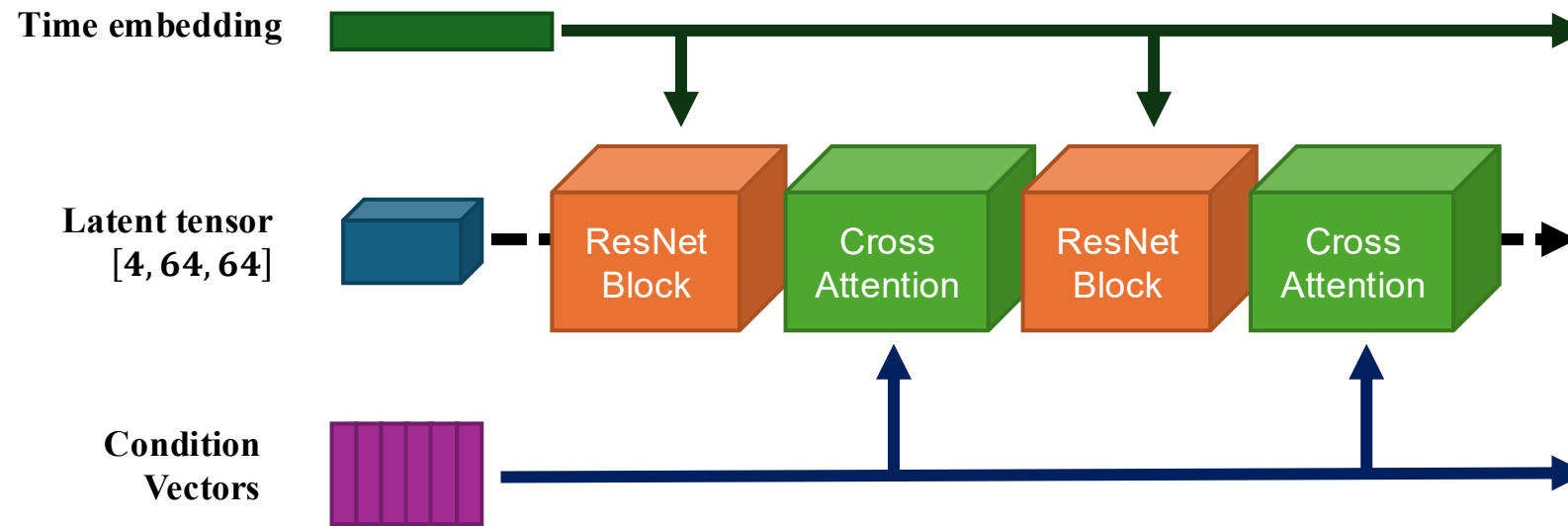
Conditioning



$$L_{LDM} := \mathbb{E}_{\mathcal{E}(x), y, \epsilon \sim \mathcal{N}(0,1), t} \left[\|\epsilon - \epsilon_{\theta}(z_t, t, \tau_{\theta}(y))\|_2^2 \right]$$

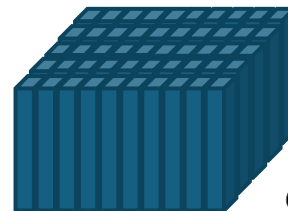
Conditioning diffusion

- Alternating ResNet Block and Cross Attention

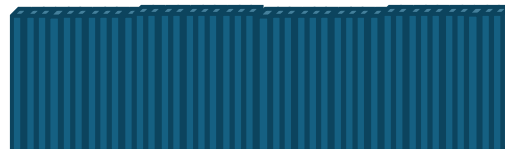


$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) \cdot V$$

$$K = W_K^{(i)} \cdot \tau_\theta(y), \quad V = W_V^{(i)} \cdot \tau_\theta(y).$$



$$Q = W_Q^{(i)} \cdot \varphi_i(z_t)$$



Code

```
class SinusoidalTimeEmbedding(nn.Module):
    """
    Standard modern diffusion timestep embedding:
    | t -> [sin(freq * t), cos(freq * t)] -> MLP
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, t):
        """
        t: (B,) int64 or float
        returns: (B, dim)
        """
        device = t.device
        half = self.dim // 2
        t = t.float()

        # Log-spaced frequencies
        freqs = torch.exp(
            -math.log(10000) * torch.arange(0, half, device=device).float() / max(half - 1, 1)
        ) # (half,)
        args = t[:, None] * freqs[None, :] # (B, half)
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=-1) # (B, 2*half)

        if self.dim % 2 == 1:
            emb = F.pad(emb, (0, 1))
        return emb
```

```
class TimeEmbeddingMLP(nn.Module):
    def __init__(self, base_dim, out_dim):
        super().__init__()
        self.time_proj = SinusoidalTimeEmbedding(base_dim)
        self.mlp = nn.Sequential(
            nn.Linear(base_dim, out_dim),
            nn.SiLU(),
            nn.Linear(out_dim, out_dim),
        )

    def forward(self, t):
        return self.mlp(self.time_proj(t)) # (B, out_dim)
```

Code

```
class TinyTextEncoder(nn.Module):
    """
    Placeholder text encoder with a Transformer.
    Interface mimics CLIP-like outputs used by LDM:
    | token_ids -> context embeddings (B, T_text, d_context)
    """
    def __init__(self, vocab_size=10000, d_model=512, n_heads=8, n_layers=2, max_len=77):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Parameter(torch.randn(1, max_len, d_model) * 0.02)
        enc_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=n_heads, dim_feedforward=4 * d_model,
            activation="gelu", batch_first=True, norm_first=True
        )
        self.encoder = nn.TransformerEncoder(enc_layer, num_layers=n_layers)
        self.ln = nn.LayerNorm(d_model)
        self.max_len = max_len
        self.d_model = d_model

    def forward(self, token_ids):
        """
        token_ids: (B, T_text)
        returns: context (B, T_text, d_model)
        """
        B, T = token_ids.shape
        assert T <= self.max_len
        x = self.token_emb(token_ids) + self.pos_emb[:, :T, :]
        x = self.encoder(x)
        x = self.ln(x)
        return x
```

Code

```
class ResBlock(nn.Module):
    """
    ResNet block with timestep conditioning (additive bias from time embedding).
    Similar spirit to LDM/SD UNet ResBlocks.
    """
    def __init__(self, in_ch, out_ch, time_dim, dropout=0.0):
        super().__init__()
        self.in_ch = in_ch
        self.out_ch = out_ch

        self.norm1 = norm_2d(in_ch)
        self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)

        self.time_mlp = TimeEmbeddingMLP(time_dim, out_ch)

        self.norm2 = norm_2d(out_ch)
        self.dropout = nn.Dropout(dropout)
        self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1)

        self.skip = nn.Conv2d(in_ch, out_ch, kernel_size=1) if in_ch != out_ch else nn.Identity()

    def forward(self, x, t_emb):
        """
        x:      (B, C, H, W)
        t_emb:  (B, time_dim)
        """
        h = self.conv1(F.silu(self.norm1(x)))
        # Inject time embedding as channel-wise bias
        h = h + self.time_mlp(t_emb)[:, :, None, None]
        h = self.conv2(self.dropout(F.silu(self.norm2(h))))
        return h + self.skip(x)
```

Code

```
class CrossAttention(nn.Module):
    """
    Cross-attention: queries from image/latent features, keys/values from text context.
    Attention(Q, K, V) = softmax(QK^T / sqrt(d)) V

    This is implemented over flattened spatial tokens.
    """
    def __init__(self, dim, context_dim, n_heads=8, head_dim=64):
        super().__init__()
        inner_dim = n_heads * head_dim
        self.n_heads = n_heads
        self.head_dim = head_dim
        self.scale = head_dim ** -0.5

        self.to_q = nn.Linear(dim, inner_dim, bias=False)
        self.to_k = nn.Linear(context_dim, inner_dim, bias=False)
        self.to_v = nn.Linear(context_dim, inner_dim, bias=False)
        self.to_out = nn.Linear(inner_dim, dim)

    def forward(self, x, context):
        """
        x:          (B, N, dim)          image tokens (N = H*W)
        context: (B, T_text, c_dim)    text embeddings
        returns: (B, N, dim)
        """
        B, N, _ = x.shape
        _, T, _ = context.shape
        H = self.n_heads
        D = self.head_dim

        q = self.to_q(x).view(B, N, H, D).transpose(1, 2) # (B, H, N, D)
        k = self.to_k(context).view(B, T, H, D).transpose(1, 2) # (B, H, T, D)
        v = self.to_v(context).view(B, T, H, D).transpose(1, 2) # (B, H, T, D)

        attn = torch.matmul(q, k.transpose(-1, -2)) * self.scale # (B,H,N,T)
        attn = attn.softmax(dim=-1)

        out = torch.matmul(attn, v) # (B,H,N,D)
        out = out.transpose(1, 2).contiguous().view(B, N, H * D) # (B,N,inner_dim)
        return self.to_out(out)
```

Code

```
class SpatialTransformerBlock(nn.Module):
    """Transformer over spatial tokens" block with self-attn + cross-attn + FFN.
    This is closer to modern LDM/Stable Diffusion style conditioning than plain conv-only U-Net.
    """
    def __init__(self, channels, context_dim, n_heads=8, head_dim=64):
        super().__init__()
        self.channels = channels
        self.norm = norm_2d(channels)
        self.proj_in = nn.Conv2d(channels, channels, kernel_size=1)

        # token-wise transformer pieces
        self.ln1 = nn.LayerNorm(channels)
        self.self_attn = CrossAttention(dim=channels, context_dim=channels, n_heads=n_heads, head_dim=head_dim)

        self.ln2 = nn.LayerNorm(channels)
        self.cross_attn = CrossAttention(dim=channels, context_dim=context_dim, n_heads=n_heads, head_dim=head_dim)

        self.ln3 = nn.LayerNorm(channels)
        self.ff = GEGLU(channels)

        self.proj_out = nn.Conv2d(channels, channels, kernel_size=1)

    def forward(self, x, context):
        """
        x: (B,C,H,W), context: (B,T_text,C_ctx)
        """
        B, C, H, W = x.shape
        x_in = x

        x = self.proj_in(self.norm(x))  # (B,C,H,W)
        x = x.permute(0, 2, 3, 1).reshape(B, H * W, C)  # (B,N,C)

        # self-attention over spatial tokens (context = x)
        x = x + self.self_attn(self.ln1(x), self.ln1(x))
        # cross-attention: queries from image tokens, keys/values from text
        x = x + self.cross_attn(self.ln2(x), context)
        # FFN
        x = x + self.ff(self.ln3(x))

        x = x.view(B, H, W, C).permute(0, 3, 1, 2).contiguous()
        x = self.proj_out(x)

        return x + x_in
```

Code

```
# ----- Encoder (down path) -----
self.down_blocks = nn.ModuleList()
ch = base_channels
self.skip_channels = [ch]

for level, mult in enumerate(channel_mults):
    out_ch = base_channels * mult
    for _ in range(num_res_blocks):
        block = nn.ModuleDict({
            "res": ResBlock(ch, out_ch, time_dim=time_emb_dim),
            "attn": SpatialTransformerBlock(
                out_ch, context_dim=context_dim, n_heads=n_heads, head_dim=head_dim
            )
        })
        self.down_blocks.append(block)
        ch = out_ch
        self.skip_channels.append(ch)

    if level != len(channel_mults) - 1:
        self.down_blocks.append(nn.ModuleDict({"downsample": Downsample(ch)}))
        self.skip_channels.append(ch)

# ----- Middle blocks -----
self.mid_block1 = ResBlock(ch, ch, time_dim=time_emb_dim)
self.mid_attn = SpatialTransformerBlock(ch, context_dim=context_dim, n_heads=n_heads, head_dim=head_dim)
self.mid_block2 = ResBlock(ch, ch, time_dim=time_emb_dim)
```

```
# ----- Decoder (up path) -----
self.up_blocks = nn.ModuleList()
skip_stack = list(self.skip_channels)

for level, mult in reversed(list(enumerate(channel_mults))):
    out_ch = base_channels * mult

    # num_res_blocks + 1 is common in UNet decoders due to skip merges
    for _ in range(num_res_blocks + 1):
        skip_ch = skip_stack.pop()
        block = nn.ModuleDict({
            "res": ResBlock(ch + skip_ch, out_ch, time_dim=time_emb_dim),
            "attn": SpatialTransformerBlock(
                out_ch, context_dim=context_dim, n_heads=n_heads, head_dim=head_dim
            ),
        })
        self.up_blocks.append(block)
        ch = out_ch

    if level != 0:
        self.up_blocks.append(nn.ModuleDict({"upsample": Upsample(ch)}))

self.out_norm = norm_2d(ch)
self.out_conv = nn.Conv2d(ch, in_channels, kernel_size=3, padding=1)
```

Code

```
def ddpm_loss(self, images, token_ids, alphaBars):  
    """  
    Standard DDPM epsilon-prediction loss, but in latent space and conditioned on text.  
    """  
  
    B = images.shape[0]  
    device = images.device  
    T = alphaBars.shape[0]  
  
    # (1) Encode image to latent space (in real LDM: usually pretrained VAE, often frozen)  
    z0 = self.encode_latents(images) # (B,4,H_lat,W_lat)  
  
    # (2) Encode text condition  
    context = self.get_text_context(token_ids) # (B,T_text,C_ctx)  
  
    # (3) Sample diffusion timestep per example  
    t = torch.randint(0, T, (B,), device=device, dtype=torch.long)  
  
    # (4) Forward noising in latent space  
    eps = torch.randn_like(z0)  
    alpha_bar_t = extract_1d_to_4d(alphaBars, t, z0.shape)  
    z_t = torch.sqrt(alpha_bar_t) * z0 + torch.sqrt(1.0 - alpha_bar_t) * eps  
  
    # (5) Predict epsilon with text-conditioned latent U-Net  
    eps_pred = self.unet(z_t, t, context)  
  
    # (6) DDPM objective (simple MSE)  
    loss = F.mse_loss(eps_pred, eps)  
    return loss
```